AI Agent Trading Bot Platform

Software Architecture Document (SAD)

---

1. Document Information

Field	Value
Project Name	AI Agent Trading Bot Platform
Version	1.0
Document Type	Software Architecture Document (SAD)
Author	Ayberk Yavuz
Date	May 2026
Target Environment	Local Mac Mini Deployment

---

2. Architecture Overview

The AI Agent Trading Bot Platform is designed as a modular, workflow-oriented AI trading system.

The architecture prioritizes:

- modularity
- observability
- cost efficiency
- deterministic risk management
- AI workflow orchestration
- maintainability
- future scalability

The system uses a hybrid architecture where deterministic software components perform repetitive calculations and AI agents provide contextual reasoning only when necessary.

---

3. High-Level Architecture

The platform consists of the following major layers:

1. Frontend Layer
2. Backend API Layer
3. Agent Orchestration Layer
4. Tool Layer
5. Database Layer
6. Third-Party Service Layer

---

4. High-Level System Architecture Diagram

The AI Agent Trading Bot Platform is designed using a modular and layered architecture approach to ensure scalability, maintainability, observability, and cost-efficient AI operations. The system separates user interaction, backend orchestration, AI agent workflows, deterministic tool execution, and external service integrations into independent architectural layers.

The React frontend acts as the user interaction layer and communicates with the FastAPI backend through REST APIs and WebSocket connections. REST APIs are used for standard operations such as authentication and configuration management, while WebSocket communication provides real-time updates for live trades, portfolio status, agent decisions, and system notifications.

The FastAPI backend serves as the central orchestration layer of the platform. It is responsible for request handling, authentication, workflow coordination, database communication, WebSocket management, and integration with external services. The backend also controls the lifecycle of AI trading workflows and ensures secure and observable system execution.

The Agent Orchestrator layer, implemented using LangGraph, manages the execution flow of AI agents. Instead of autonomous recursive agents, the platform uses workflow-oriented agents with clearly defined responsibilities and deterministic execution boundaries. This architecture improves system reliability, explainability, and operational safety while reducing unnecessary LLM API costs.

The Tool Layer provides abstraction between AI agents and external systems. Agents interact with deterministic tools rather than directly communicating with APIs or infrastructure components. The tool layer includes components such as Binance Tool, Indicator Tool, News Tool, and Risk Tool. This separation improves modularity, maintainability, and testing capabilities.

PostgreSQL is used as the relational database management system for storing user information, transaction records, agent decisions, LLM usage logs, and portfolio snapshots. Structured logging and persistent storage provide auditability and support future analytical capabilities.

External services include Binance APIs for market data retrieval and trade execution, and LLM APIs for sentiment analysis and contextual reasoning tasks. The system architecture intentionally limits LLM usage to specific reasoning workflows in order to maintain operational cost efficiency.

Figure 1 illustrates the high-level architecture and communication flow between system components.

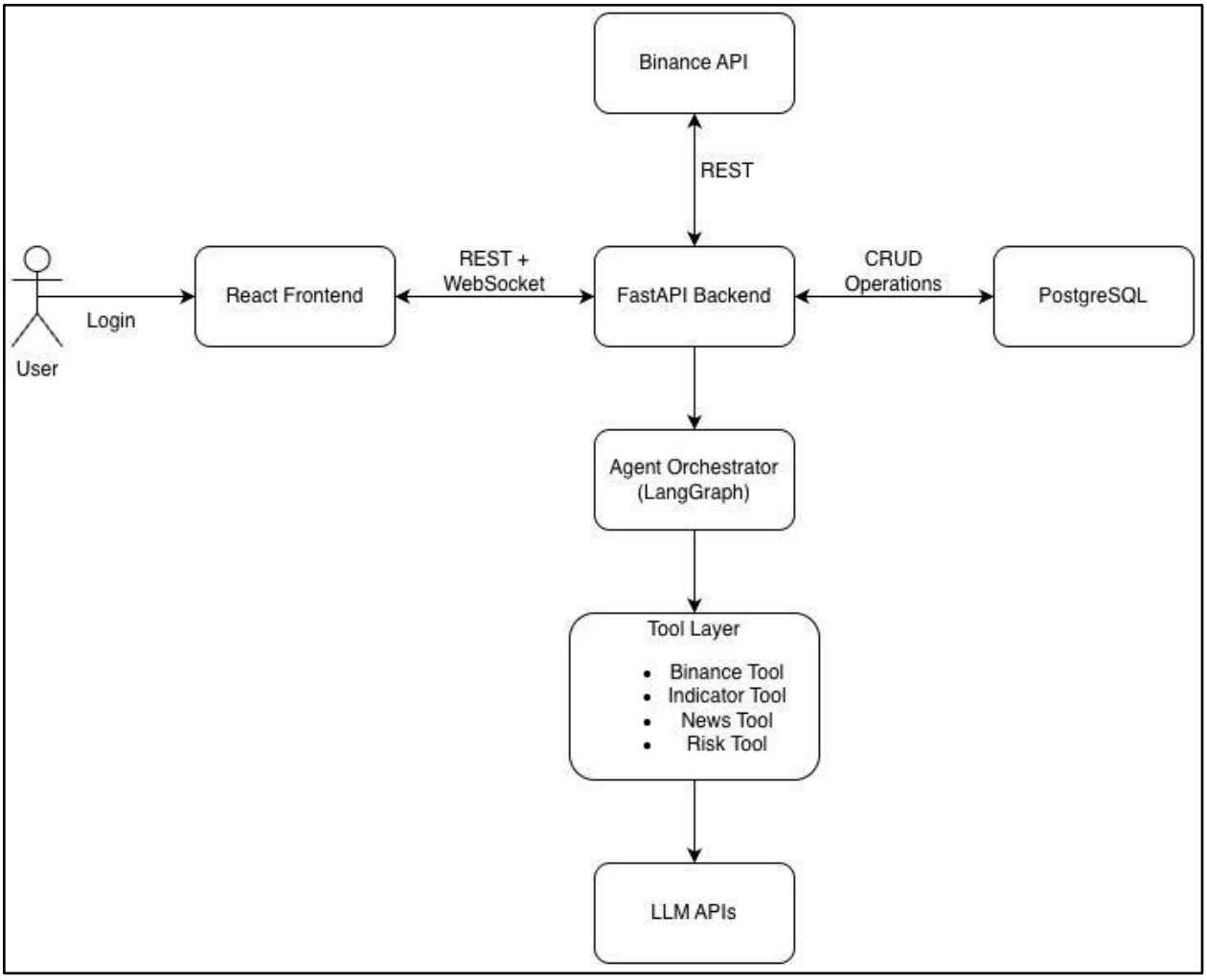

Figure 1

5. Frontend Architecture

5.1 Overview

The frontend is responsible for:

- user authentication
- dashboard rendering
- live transaction updates
- agent monitoring
- portfolio visualization
- manual trading controls

The frontend communicates with FastAPI backend using:

- REST APIs
- WebSocket connections

5.2 Frontend Technology Stack

Component	Technology
Framework	React
Build Tool	Vite
Styling	Tailwind CSS
UI Components	shadcn/ui
Real-Time Communication	WebSockets

5.3 Frontend Pages

Login Page

Responsibilities:

- user authentication
- session creation

Dashboard Page

Responsibilities:

- start/stop trading
- display live trades
- display PnL
- display portfolio
- display agent status

Trade History Page

Responsibilities:

- show historical trades
- show transaction details
- show profit/loss history

Agent Logs Page

Responsibilities:

- display AI reasoning summaries
- display agent outputs
- display system events

---

6. Backend Architecture

6.1 Overview

The backend acts as the central coordination layer.

Responsibilities:

- API management
- authentication
- agent orchestration
- trade execution
- WebSocket management
- database communication
- logging

- scheduling

6.2 Backend Technology Stack

Component	Technology
Framework	FastAPI
Language	Python
ORM	SQLAlchemy
Migration Tool	Alembic
Real-Time Communication	WebSockets
Logging	structlog

6.3 Backend Module Structure

```
backend/
│
├── api/
├── agents/
├── workflows/
├── tools/
├── services/
├── db/
├── models/
├── schemas/
├── configs/
├── logs/
└── tests/
```

---

7. Agent Architecture

7.1 Architecture Philosophy

The system uses workflow-oriented AI agents instead of autonomous recursive agents.

Key principles:

- deterministic workflows
- constrained responsibilities
- structured outputs
- observable reasoning

- event-driven execution
- cost-efficient AI usage

7.2 Agent Overview

The system contains four primary agents:

1. Market Scanner Agent
2. News & Sentiment Agent
3. Trade Decision Agent
4. Risk Manager Agent

7.3 Market Scanner Agent

Responsibilities

- monitor market data
- calculate technical indicators
- detect trading opportunities
- identify volatility spikes

Inputs

- Binance market data
- historical price data
- technical indicators

Outputs

```
{
  "symbol": "BTCUSDT",
  "signal_strength": 0.78,
  "reason": "Volume spike + RSI oversold"
}
```

Architecture Characteristics

- deterministic processing
- minimal LLM usage
- low operational cost

7.4 News & Sentiment Agent

Responsibilities

- retrieve crypto news
- analyze sentiment
- interpret macro-level events

Inputs

- crypto news
- market summaries
- geopolitical summaries

Outputs

```
{
  "market_sentiment": "bullish",
  "confidence": 0.67,
  "reason": "Positive ETF inflow signals"
}
```

Architecture Characteristics

- LLM-assisted analysis
- structured outputs
- event-driven execution

7.5 Trade Decision Agent

Responsibilities

- combine technical and sentiment signals
- produce BUY/SELL/HOLD decisions
- generate confidence scores

Inputs

- scanner outputs
- sentiment outputs
- portfolio state
- risk constraints

Outputs

```
{
  "decision": "BUY",
  "symbol": "BTCUSDT",
  "confidence": 0.82,
  "reason": "Strong momentum with bullish sentiment"
}
```

Architecture Characteristics

- structured prompts
- JSON outputs only
- controlled reasoning

7.6 Risk Manager Agent

Responsibilities

- validate trade safety
- enforce risk limits
- stop dangerous trades

Example Rules

- max daily loss
- max open positions
- max trades per hour
- duplicate trade prevention

Architecture Characteristics

- deterministic rules
- highest execution priority
- minimal or no LLM usage

---

8. Workflow Architecture

8.1 Trading Workflow

The AI Agent Trading Bot Platform uses an event-driven workflow architecture designed to minimize unnecessary AI processing while maintaining responsive and intelligent trading behavior. The workflow combines deterministic market analysis with selective AI reasoning and risk-controlled trade execution.

The workflow begins with a scheduler trigger that periodically activates the market evaluation process. Instead of continuously invoking AI agents, the system first performs market event detection to identify meaningful market conditions such as volatility spikes, momentum changes, abnormal volume activity, or other predefined trading signals.

When a meaningful market event is detected, the Market Scanner Agent analyzes market conditions using deterministic technical indicators and filtering logic. This layer performs low-cost computations such as RSI calculations, moving averages, momentum analysis, and volatility checks without requiring LLM usage.

The News & Sentiment Agent is then activated to analyze market-related news, sentiment trends, and contextual information using LLM APIs. This agent provides structured sentiment analysis outputs that support higher-level market interpretation.

The Trade Decision Agent combines outputs from technical analysis and sentiment analysis to generate BUY, SELL, or HOLD decisions together with confidence scores and reasoning summaries. The workflow intentionally uses structured JSON outputs to improve reliability and reduce token consumption.

Before any trade execution occurs, the workflow performs a Confidence Threshold Check. Low-confidence decisions are ignored to reduce overtrading and unnecessary risk exposure. Only sufficiently confident decisions proceed to the Risk Manager Agent.

The Risk Manager Agent applies deterministic validation rules such as maximum daily loss limits, position sizing constraints, and duplicate trade prevention checks. This layer acts as the final safety mechanism before financial execution.

After successful risk validation, the Trade Executor Service places orders through Binance APIs and verifies execution status. All transactions, decisions, and operational logs are then persisted into PostgreSQL for auditability and future analysis.

Finally, the system pushes live updates to the frontend dashboard through WebSocket communication, allowing the user to monitor trades, portfolio status, and agent activities in real time.

Figure 2 illustrates the end-to-end AI trading workflow of the platform.

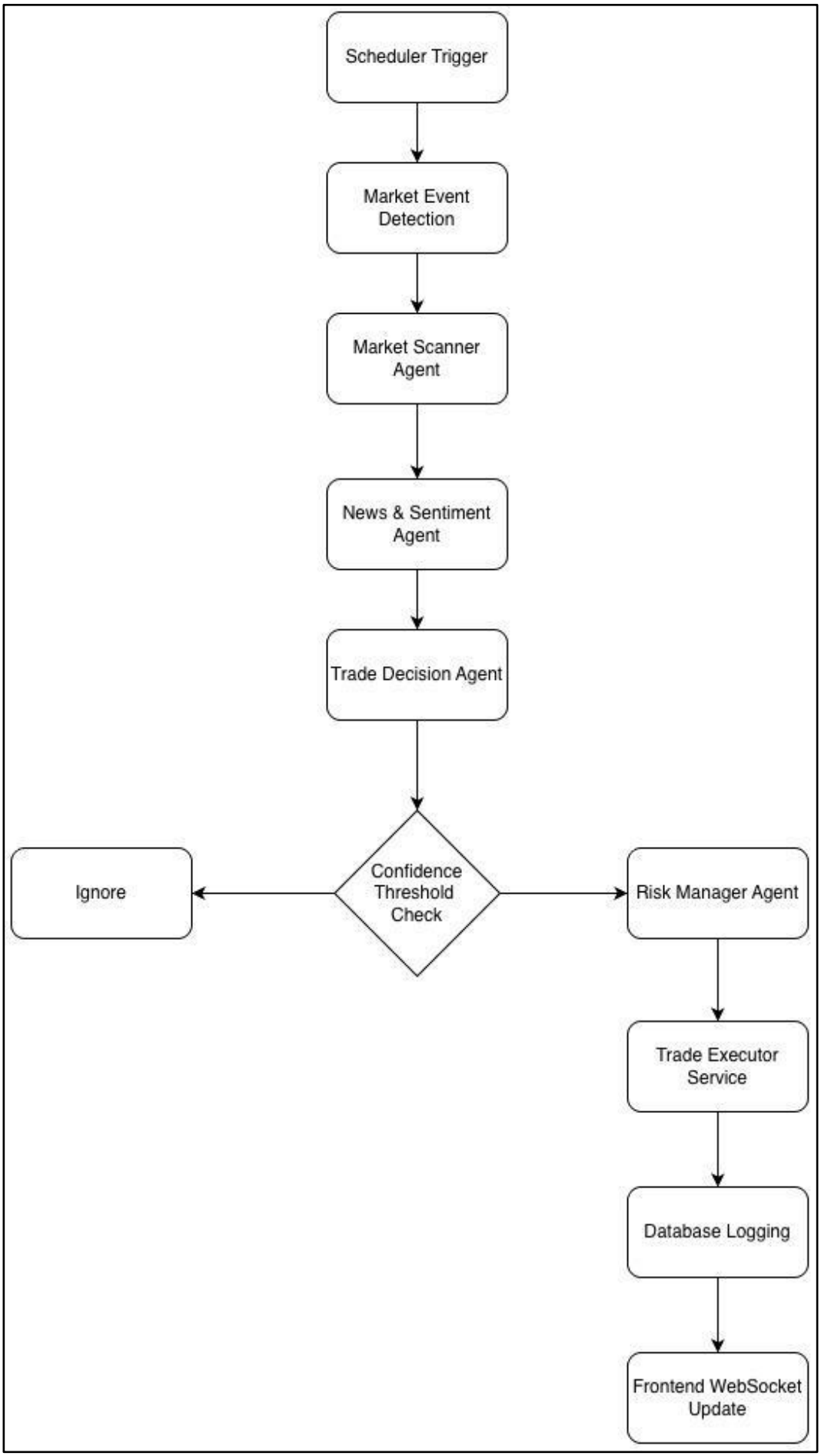
Scheduler Trigger
Market Event Detection
Market Scanner Agent
News & Sentiment Agent
Trade Decision Agent
Confidence Threshold Check
Ignore
Risk Manager Agent
Trade Executor Service
Database Logging
Frontend WebSocket Update

Figure 2

8.2 Event-Driven Execution

The system uses event-driven AI execution.

AI reasoning is triggered only during:

- volatility spikes
- strong technical signals
- major news events
- unusual market movements

This architecture minimizes unnecessary LLM costs.

---

9. Tool Layer Architecture

9.1 Overview

The tool layer provides deterministic functionality to AI agents.

Agents do not directly interact with external services.

Instead, all interactions occur through controlled tools.

9.2 Binance Tool

Responsibilities:

- retrieve market data
- place buy/sell orders
- retrieve balances
- monitor order status

9.3 Technical Indicator Tool

Responsibilities:

- RSI calculation
- MACD calculation
- moving averages
- momentum analysis
- volatility analysis

9.4 News Tool

Responsibilities:

- retrieve crypto news
- preprocess articles
- generate compact summaries

9.5 Portfolio Tool

Responsibilities:

- retrieve balances
- calculate PnL
- monitor open positions

9.6 Risk Tool

Responsibilities:

- validate position sizes
- enforce trading limits
- stop excessive losses

---

10. Trade Executor Service Architecture

10.1 Overview

Trade execution is implemented as a deterministic backend service.

It is intentionally separated from AI agents.

10.2 Responsibilities

- validate orders
- place Binance orders
- verify execution status
- handle retries
- store transaction records
- publish frontend updates

---

11. Database Architecture

11.1 Database Technology

Component	Technology
Database	PostgreSQL
ORM	SQLAlchemy
Migration Tool	Alembic

11.2 Database Tables

users

Field	Type
id	UUID
email	VARCHAR
password_hash	VARCHAR
created_at	TIMESTAMP

---

trades

Field	Type
id	UUID
symbol	VARCHAR
action	VARCHAR
quantity	FLOAT

Field	Type
price	FLOAT
profit_loss	FLOAT
timestamp	TIMESTAMP

---

agent_decisions

Field	Type
id	UUID
agent_name	VARCHAR
decision	VARCHAR
confidence	FLOAT
reason	TEXT
created_at	TIMESTAMP

---

llm_logs

Field	Type
id	UUID
model_name	VARCHAR
prompt_tokens	INTEGER
completion_tokens	INTEGER
cost	FLOAT
latency	FLOAT
created_at	TIMESTAMP

---

portfolio_snapshots

Field	Type
id	UUID
balance	FLOAT
daily_pnl	FLOAT
open_positions	JSONB
created_at	TIMESTAMP

---

12. Real-Time Communication Architecture

12.1 WebSocket Architecture

The frontend maintains persistent WebSocket connections with FastAPI backend.

Responsibilities:

- live trade updates
- live PnL updates
- agent status updates
- error notifications
- cost monitoring updates

12.2 WebSocket Events

Trade Executed Event

```
{
  "event": "trade_executed",
  "symbol": "BTCUSDT",
  "action": "BUY"
}
```

Agent Decision Event

```
{
  "event": "agent_decision",
  "agent": "Trade Decision Agent"
}
```

---

13. Third-Party Service Architecture

13.1 Binance APIs

Responsibilities:

- market data
- order execution
- balance retrieval
- order tracking

13.2 OpenAI APIs

Responsibilities:

- sentiment analysis
- contextual reasoning
- decision support

13.3 OpenRouter

Responsibilities:

- alternative model routing
- model experimentation
- failover support

---

14. Deployment Architecture

14.1 Deployment Environment

Initial deployment target:

- Mac Mini
- Docker Compose based local environment

14.2 Docker Services

```
+-------------------------+
| frontend container      |
+-------------------------+

+-------------------------+
| backend container       |
+-------------------------+

+-------------------------+
| postgres container      |
+-------------------------+
```

14.3 Future Cloud Migration

Future architecture may include:

- Kubernetes
- Redis
- Prometheus
- Grafana
- CI/CD pipelines

---

15. Logging & Observability Architecture

15.1 Logging Principles

The system uses structured JSON logging.

Responsibilities:

- track AI decisions
- track errors
- track trade execution
- track API costs
- support debugging

15.2 Logged Data

The system shall log:

- prompts
- model responses
- token usage
- API latency
- trade decisions
- trade execution results

- risk validation outcomes

---

16. Security Architecture

16.1 Authentication Security

The system shall:

- hash passwords securely
- validate sessions
- restrict unauthorized access

16.2 API Key Management

API keys shall:

- remain outside source control
- use environment variables
- remain encrypted where applicable

16.3 Trading Safety

The system shall:

- enforce risk controls
- validate balances
- support emergency stop mechanisms

---

17. Cost Optimization Architecture

17.1 AI Cost Optimization Principles

The architecture minimizes AI costs using:

- deterministic filtering
- event-driven reasoning

- cached AI outputs
- lightweight models
- structured prompts
- JSON-only outputs

17.2 Three-Layer Intelligence Architecture

Layer 1 — Deterministic Filtering

Responsibilities:

- technical indicators
- volatility checks
- signal filtering

LLM usage:

- none

Layer 2 — Lightweight AI Analysis

Responsibilities:

- sentiment analysis
- event interpretation

LLM usage:

- lightweight models

Layer 3 — Premium Reasoning

Responsibilities:

- conflict resolution
- high-confidence trade analysis

LLM usage:

- stronger models

- very limited frequency

---

18. API Contracts

18.1 Overview

The AI Agent Trading Bot Platform exposes REST APIs and WebSocket endpoints for frontend communication, workflow management, monitoring, and real-time dashboard updates.

The API architecture follows the following principles:

- REST-based backend communication
- JSON request/response payloads
- stateless API design
- structured error handling
- real-time WebSocket updates
- secure authenticated endpoints
- modular endpoint grouping

The FastAPI backend acts as the single entry point for all frontend and external communication.

18.2 API Base URL

http://localhost:8000/api

18.3 Authentication Strategy

Authentication will use JWT-based access tokens.

Protected endpoints require:

Authorization: Bearer <jwt_token>

Passwords shall be securely hashed before persistence.

18.4 Standard Response Format

All REST APIs should return structured JSON responses.

Success Response


```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```


18.5 Error Response


```
{
  "success": false,
  "message": "Validation error.",
  "error_code": "INVALID_REQUEST"
}
```


18.6 Authentication APIs

18.6.1 User Login

Endpoint

POST /api/auth/login

Description

Authenticates the user and generates a JWT access token.

Request Body


```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response


```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "access_token": "jwt_token_here",
    "token_type": "bearer"
  }
}
```


18.6.2 User Logout

Endpoint

POST /api/auth/logout

Description

Terminates active user sessions.

Response


```
{
  "success": true,
  "message": "Logout successful."
}
```


18.7 Trading Control APIs

18.7.1 Start Trading Workflow

Endpoint

POST /api/trading/start

Description

Starts the AI trading workflow and activates market monitoring.

Request Body


```
{
  "symbols": [
    "BTCUSDT",
    "ETHUSDT"
  ],
  "initial_budget": 200,
  "max_daily_loss": 20,
  "trading_mode": "paper"
}
```


Request Parameters Description

Field	Description
symbols	Trading pairs to monitor
initial_budget	Maximum usable trading budget
max_daily_loss	Daily stop-loss threshold
trading_mode	paper or live

Response


```
{
  "success": true,
  "message": "Trading workflow started successfully.",
  "data": {
    "workflow_status": "active"
  }
}
```

Example Invalid Request

{

"symbols": [],

"initial_budget": 5,

"max_daily_loss": 50,

"trading_mode": "invalid_mode"

}

Example Validation Error Response

{

"success": false,

"message": "Validation error.",

"error_code": "INVALID_TRADING_CONFIGURATION"

}

18.7.2 Stop Trading Workflow

Endpoint

POST /api/trading/stop

Description

Stops all active trading workflows and disables further trade execution.

Response

{

"success": true,

"message": "Trading workflow stopped successfully."

}

18.7.3 Emergency Kill Switch

Endpoint

POST /api/trading/emergency-stop

Description

Immediately stops all active workflows and prevents all trading operations.

Response

{

"success": true,

"message": "Emergency stop activated."

}

18.7.4 Update Risk Limits

Endpoint

PUT /api/trading/risk-limits

Description

Updates runtime trading risk parameters without restarting the trading workflow.

This endpoint allows dynamic adjustment of operational risk controls while agents continue running.

Request Body

{

"max_daily_loss": 20,

"max_position_size": 50

}

Validation Rules

Field	Validation
max_daily_loss	Must be greater than 0
max_position_size	Must be greater than 0

Response

{

"success": true,

"message": "Risk limits updated successfully."

}

18.7.5 Switch Paper/Live Trading Mode

Endpoint

POST /api/trading/paper-live

Description

Switches the active trading mode between paper trading and live trading without restarting the workflow.

Request Body

{

"trading_mode": "paper"

}

Allowed Values

Value	Description
paper	Simulated trading
live	Real market execution

Response

{

"success": true,

"message": "Trading mode updated successfully.",

"data": {

"trading_mode": "paper"

}

}

18.7.6 Manual Position Close

Endpoint

DELETE /api/trades/open-positions/{symbol}

Description

Closes an active position manually.

This endpoint is designed for emergency intervention, operational overrides, or manual trade exits.

Path Parameters

Parameter	Description
symbol	Trading pair symbol

Example

DELETE /api/trades/open-positions/BTCUSDT

Response


```
{
 "success": true,
 "message": "Position closed successfully.",
 "data": {
   "symbol": "BTCUSDT",
   "exit_price": 64620
 }
}
```


18.8 Portfolio APIs

18.8.1 Get Portfolio Status

Endpoint

GET /api/portfolio

Description

Retrieves current portfolio information.

Response


```
{
```

```
  "success": true,
  "data": {
    "total_balance": 198.42,
    "available_cash": 124.18,
    "total_invested": 74.24,
    "daily_pnl": 3.15,
    "win_rate": 0.64,
    "sharpe_ratio": 1.42,
    "open_positions": [
      {
        "symbol": "BTCUSDT",
        "quantity": 0.002,
        "entry_price": 64200,
        "current_price": 64550,
        "unrealized_pnl": 0.70
      }
    ]
  }
}
```


18.8.2 Get Portfolio History

Endpoint

GET /api/portfolio/history

Description

Retrieves historical portfolio snapshots.

Response


```
{
```

```
 "success": true,
 "data": [
  {
    "timestamp": "2026-05-12T10:00:00",
    "balance": 200.00
  },
  {
    "timestamp": "2026-05-12T11:00:00",
    "balance": 202.15
  }
 ]
}
```


18.9 Trade Monitoring APIs

18.9.1 Get Trade History

Endpoint

GET /api/trades

Description

Retrieves historical trade records.

Query Parameters

Parameter	Description
limit	Number of records
symbol	Filter by trading pair
start_date	Start date filter
end_date	End date filter

Example Request

GET /api/trades?symbol=BTCUSDT&limit=100&start_date=2026-05-12T00:00:00Z&end_date=2026-05-13T00:00:00Z

Response


```
{
 "success": true,
 "data": [
   {
     "trade_id": "uuid",
     "symbol": "BTCUSDT",
     "action": "BUY",
     "quantity": 0.002,
     "price": 64200,
     "timestamp": "2026-05-12T12:15:00"
   }
 ]
}
```


18.9.2 Get Open Positions

Endpoint

GET /api/trades/open-positions

Description

Retrieves currently active positions.

Response


```
{
 "success": true,
 "data": [
```

```
  {
    "symbol": "BTCUSDT",
    "quantity": 0.002,
    "entry_price": 64200,
    "current_price": 64550,
    "unrealized_pnl": 0.7
  }
 ]
}
```


18.10 Agent Monitoring APIs

18.10.1 Get Agent Status

Endpoint

GET /api/agents/status

Description

Retrieves active agent statuses.

Response


```
{
 "success": true,
 "data": {
   "market_scanner_agent": "active",
   "news_sentiment_agent": "active",
   "trade_decision_agent": "active",
   "risk_manager_agent": "active"
 }
}
```

18.10.2 Get Agent Decision Logs

Endpoint

GET /api/agents/decisions

Description

Retrieves historical AI agent decisions.

Response


```
{
 "success": true,
 "data": [
  {
   "agent_name": "Trade Decision Agent",
   "decision": "BUY",
   "confidence": 0.81,
   "reason": "Bullish momentum and positive sentiment.",
   "timestamp": "2026-05-12T12:20:00"
  }
 ]
}
```


18.11 LLM Monitoring APIs

18.11.1 Get LLM Usage Metrics

Endpoint

GET /api/llm/usage

Description

Retrieves LLM API usage statistics and operational costs.

Response

```
{
 "success": true,
 "data": {
   "daily_token_usage": 18234,
   "daily_cost": 1.82,
   "average_latency_ms": 842
 }
}
```


18.11.2 LLM Cost Breakdown

Endpoint

GET /api/llm/costs/breakdown

Description

Retrieves detailed LLM API usage costs grouped by AI agent.

This endpoint helps optimize operational cost efficiency and identify expensive workflows.

Response


```
{
 "success": true,
 "data": {
   "market_scanner_agent": {
     "daily_tokens": 1200,
     "daily_cost": 0.08
   },
   "news_sentiment_agent": {
```

```
    "daily_tokens": 18200,
    "daily_cost": 1.74
  },
  "trade_decision_agent": {
    "daily_tokens": 6400,
    "daily_cost": 0.63
  }
 }
}
```


18.12 System Health Check APIs

18.12.1 Health Check

Endpoint

GET /api/health

Description

Checks system availability and service status.

Response


```
{
 "success": true,
 "data": {
  "backend": "healthy",
  "postgresql": "healthy",
  "binance_api": "healthy",
  "llm_api": "healthy"
 }
}
```

18.13 WebSocket Contract

18.13.1 WebSocket Endpoint

Endpoint

WS /ws/dashboard

Description

Provides real-time updates for dashboard events.

18.13.2 Trade Executed Event


```
{
  "event_type": "trade_executed",
  "symbol": "BTCUSDT",
  "action": "BUY",
  "quantity": 0.002,
  "price": 64200,
  "timestamp": "2026-05-12T12:25:00"
}
```


18.13.3 Agent Decision Event


```
{
  "event_type": "agent_decision",
  "agent_name": "Trade Decision Agent",
  "decision": "BUY",
  "confidence": 0.82
}
```


18.13.4 Portfolio Update Event


```
{
  "event_type": "portfolio_update",
  "total_balance": 203.14,
```

```
  "daily_pnl": 3.14
}
```


18.13.5 Error Notification Event


```
{
  "event_type": "system_error",
  "service": "Binance API",
  "message": "Temporary API timeout."
}
```


18.14 API Security Considerations

The API layer shall implement the following security mechanisms:

- JWT-based authentication
- protected endpoints
- request validation using Pydantic
- secure password hashing
- environment variable-based secret management
- rate limiting (future phase)
- structured audit logging

18.15 API Design Principles

The API architecture follows the following engineering principles:

- modular endpoint grouping
- stateless REST architecture
- consistent JSON response structures
- real-time WebSocket updates
- observable API operations
- clear separation of concerns
- future scalability support

The API layer is intentionally designed to support future cloud-native deployment and multi-user expansion scenarios.

---

19. Testing Strategy

19.1 Overview

The AI Agent Trading Bot Platform adopts a safety-first and reliability-oriented testing strategy designed to minimize operational risk, validate AI workflows, and ensure stable trading behavior before real-money execution.

The testing architecture combines deterministic software testing methodologies with AI workflow validation, paper trading simulations, and controlled production rollout phases. The system intentionally separates business logic, AI reasoning, and external integrations to improve testability and reduce operational uncertainty.

The testing strategy prioritizes the following engineering goals:

- deterministic validation of critical workflows
- safe financial system behavior
- controlled AI execution
- operational observability
- cost-efficient testing
- dependency isolation
- incremental deployment confidence

The platform shall follow a phased validation process before enabling unrestricted real-money trading.

19.2 Testing Layers

The testing lifecycle is organized into multiple validation layers.

Unit Tests

↓

Integration Tests

↓

Workflow Tests

↓

Paper Trading Simulation

↓

Limited Real Trading

Each testing layer validates different aspects of the system and progressively increases operational realism.

19.3 Unit Testing Strategy

Unit tests validate isolated business logic and deterministic system behavior without requiring external dependencies.

The following components shall be covered with unit tests:

- Binance Tool
- Indicator Tool
- Risk Tool
- Portfolio calculations
- trade validation logic
- confidence threshold logic
- API request validation
- WebSocket event formatting
- database repository methods

The platform shall use `pytest` as the primary testing framework.

Unit tests should remain deterministic and avoid direct external API calls whenever possible.

Mocking strategies shall be used for:

- Binance APIs
- LLM APIs
- database operations
- WebSocket communication

The unit testing layer aims to provide fast and low-cost validation of core business logic.

19.4 Integration Testing Strategy

Integration tests validate communication between system components and ensure interoperability across the platform architecture.

The following integrations shall be tested:

- FastAPI ↔ PostgreSQL
- FastAPI ↔ Binance API
- FastAPI ↔ WebSocket layer
- Agent Orchestrator ↔ Tool Layer
- Tool Layer ↔ LLM APIs

Integration tests verify:

- API contract consistency
- database persistence behavior
- workflow state transitions
- transaction logging
- authentication flows
- runtime configuration updates

The integration testing layer ensures that independently tested modules operate correctly when combined together.

19.5 Workflow Testing Strategy

Workflow testing validates the correctness of AI agent orchestration and trading execution pipelines.

The following workflow scenarios shall be tested:

- market event detection
- confidence threshold filtering
- risk validation blocking
- duplicate trade prevention
- successful trade execution
- trade rejection scenarios
- emergency stop behavior
- runtime configuration updates

Workflow tests shall validate:

- correct execution order
- expected state transitions
- deterministic safety validations
- structured JSON outputs
- failure recovery behavior

The workflow testing layer is critical because the platform relies heavily on orchestrated AI execution pipelines rather than isolated AI calls.

19.6 MockBinanceTool Strategy

The platform shall implement a dedicated `MockBinanceTool` component for safe local testing and simulation.

The MockBinanceTool shall simulate:

- account balances
- market prices
- order execution
- partial fills
- failed executions
- trading fees
- slippage scenarios

The mock implementation enables developers to validate trading workflows without interacting with live financial markets.

The MockBinanceTool provides several advantages:

- zero financial risk
- deterministic execution
- reproducible testing
- offline development support
- rapid debugging capabilities

The mock trading environment shall be used extensively during early development phases.

19.7 FakeLLM Strategy

The platform shall implement a `FakeLLM` component to isolate AI workflow testing from real LLM API dependencies.

The FakeLLM component shall support deterministic AI response simulation for:

- bullish sentiment outputs
- bearish sentiment outputs
- neutral sentiment outputs
- malformed JSON responses
- timeout scenarios
- latency simulation
- low-confidence outputs
- invalid reasoning structures

Example FakeLLM response:

{

"decision": "BUY",

"confidence": 0.82,

"reason": "Positive momentum and bullish market sentiment detected."

}

The FakeLLM strategy provides the following benefits:

- reduced LLM API costs
- deterministic workflow testing
- reproducible debugging
- failure scenario validation
- offline development support

The FakeLLM layer is especially important for validating workflow orchestration and structured response parsing logic.

19.8 Paper Trading Simulation

Before enabling real-money trading, the platform shall operate in paper trading mode for a controlled validation period.

The paper trading phase shall use:

Binance Spot Testnet

The simulation phase aims to validate:

- trading workflow stability
- AI decision consistency
- order execution logic
- profit/loss calculations
- WebSocket event delivery
- logging accuracy
- runtime risk management

Recommended paper trading duration:

1–2 weeks

No real financial transactions shall occur during this phase.

Paper trading performance metrics shall be monitored continuously before production rollout approval.

19.9 Limited Real Trading Rollout

After successful paper trading validation, the system shall transition into limited real-money trading mode.

Initial rollout constraints:

Parameter	Value
Initial Capital	200 USD
Trading Frequency	Low
Position Size	Small
Manual Monitoring	Enabled
Risk Limits	Strict

The limited rollout strategy minimizes operational risk while validating real-market behavior.

The initial production phase aims to evaluate:

- real-world latency
- exchange execution behavior
- LLM operational costs
- AI decision quality
- trading profitability
- system reliability

This phase also enables incremental tuning of risk parameters and workflow optimizations.

19.10 Observability & Monitoring Validation

The testing strategy includes validation of operational observability mechanisms.

The following monitoring features shall be tested:

- structured logging
- WebSocket event delivery
- transaction persistence
- AI decision logging
- runtime metrics collection
- LLM token usage tracking

- per-agent cost analytics
- error notifications

The platform shall maintain complete auditability of all trading activities and AI decisions.

Observability validation is especially important for debugging AI workflows and monitoring operational safety.

19.11 Failure Scenario Testing

The platform shall explicitly test abnormal operational scenarios and infrastructure failures.

Failure scenarios include:

- Binance API downtime
- LLM API timeout
- malformed AI responses
- invalid JSON outputs
- database connectivity failures
- WebSocket disconnections
- duplicate order attempts
- partial trade executions
- delayed market responses

Failure testing validates:

- graceful degradation
- retry behavior
- timeout handling
- error logging
- operational recovery mechanisms

The platform shall prioritize safe failure behavior over aggressive autonomous execution.

19.12 Safety & Risk Validation Principles

The platform follows several core operational safety principles:

- AI agents shall not directly execute trades without deterministic validation.
- Risk management rules shall override AI-generated decisions.
- Emergency stop mechanisms shall always remain available.
- Runtime operational changes shall be auditable.
- High-confidence thresholds shall be enforced before execution.

- Real-money rollout shall occur incrementally.

The system intentionally prioritizes operational safety, observability, and controlled execution over aggressive autonomous trading behavior.

19.13 Testing Strategy Conclusion

The AI Agent Trading Bot Platform adopts a layered and production-oriented testing strategy that combines deterministic software engineering practices with AI workflow validation and controlled financial system rollout procedures.

The testing architecture is intentionally designed to support:

- safe AI experimentation
- low-risk financial automation
- reproducible workflow validation
- operational observability
- future scalability

This approach enables iterative improvement of AI trading workflows while minimizing operational and financial risks during development and deployment phases.

---

20. Conclusion

The AI Agent Trading Bot Platform architecture is designed to provide:

- practical AI agent engineering experience
- cost-efficient AI workflows
- modular backend architecture
- production-grade observability
- deterministic trading safety
- future scalability

The system intentionally separates:

- AI reasoning
- deterministic computation
- financial execution

This architecture supports maintainability, observability, and controlled AI behavior while remaining suitable for future expansion into cloud-native and multi-market environments.